Teaching the Fixed-Branch Zhao–Cui Tensor-Train Filter

BayesFilter P14 Pedagogical Mathematical Note

2026-05-31

Abstract

This note retells the fixed-branch Zhao–Cui tensor-train filtering construction as a mathematical teaching document. It is written for a scientific reader who is comfortable with probability densities, Gaussian likelihoods, integrals, least squares, and matrix factorizations, but who has not previously worked with tensor-train filtering. The presentation keeps the P12–P13 mathematical content: the fixed-branch squared-TT recursion produces normalized approximate filters, and its analytical gradient differentiates the same approximate likelihood scalar computed by the forward pass. The difference is pedagogical. Tensor trains are introduced first as low-rank coupling in coefficient arrays; the filtering construction is derived before the source papers are cited; and the gradient proof is split into layers so that each derivative has a visible role in the filtering algorithm.

1 Filtering Objects Before Tensor Notation

The mathematical object of interest is not a tensor train. It is the unnormalized density that appears in one Bayesian filtering step. Let

$$x_t \in \mathbb{R}^{d_x}, \qquad \vartheta \in \mathbb{R}^{d_\vartheta}, \qquad y_t \in \mathbb{R}^{d_y} \tag{1}$$

denote the current state, a static parameter, and the observation. A dominated state-space model supplies

$$p_0(x_0, \vartheta), \qquad f_\alpha(x_t \mid x_{t-1}, \vartheta), \qquad g_\alpha(y_t \mid x_t, \vartheta), \tag{2}$$

where α is the external parameter with respect to which we may later differentiate a likelihood.

Assume that, after observing $y_{1:t-1}$, the filter has density

$$p_{t-1}(x_{t-1}, \vartheta) = p(x_{t-1}, \vartheta \mid y_{1:t-1}). \tag{3}$$

Prediction and update can be written as

$$p(x_t, \vartheta \mid y_{1:t-1}) = \int f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta)\, \mathrm{d}x_{t-1}, \tag{4}$$

$$p_t(x_t, \vartheta) = \frac{g_\alpha(y_t \mid x_t, \vartheta) p(x_t, \vartheta \mid y_{1:t-1})}{p_\alpha(y_t \mid y_{1:t-1})}. \tag{5}$$

Now introduce the enlarged variable

$$u_t = (x_t, \vartheta, x_{t-1}) \tag{6}$$

and define

$$q_t(u_t; \alpha) = p_{t-1}(x_{t-1}, \vartheta) f_\alpha(x_t \mid x_{t-1}, \vartheta) g_\alpha(y_t \mid x_t, \vartheta). \tag{7}$$

Its integral is

$$Z_t(\alpha) = \int q_t(x_t, \vartheta, x_{t-1}; \alpha)\, \mathrm{d}x_t\, \mathrm{d}\vartheta\, \mathrm{d}x_{t-1}. \tag{8}$$

Substituting (7) and integrating over x_{t-1} first gives

$$\begin{aligned} Z_t(\alpha) &= \int g_\alpha(y_t \mid x_t, \vartheta) \left[\int f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta)\, \mathrm{d}x_{t-1} \right] \mathrm{d}x_t\, \mathrm{d}\vartheta \\ &= \int g_\alpha(y_t \mid x_t, \vartheta) p(x_t, \vartheta \mid y_{1:t-1})\, \mathrm{d}x_t\, \mathrm{d}\vartheta = p_\alpha(y_t \mid y_{1:t-1}). \end{aligned} \tag{9}$$

The updated filter is therefore

$$p_t(x_t, \vartheta) = \int \frac{q_t(x_t, \vartheta, x_{t-1}; \alpha)}{Z_t(\alpha)}\, \mathrm{d}x_{t-1}. \tag{10}$$

Why this matters. Every approximation in this note is an approximation to the three operations in (8)–(10): represent q_t, integrate it, and marginalize the old state. Tensor trains are useful only if they make those three operations feasible in high dimension.

Failure mode. If the approximation to q_t is poor in regions that contribute meaningful mass to the integral, the output can be normalized but inaccurate. Normalized does not mean close to the exact posterior.

2 Scalar Nonlinear Filtering Example

Before using tensor notation, consider the scalar nonlinear model

$$x_t = \rho x_{t-1} + \eta_t, \qquad \eta_t \sim N(0, \sigma_\eta^2), \tag{11}$$

$$y_t = x_t^2 + \epsilon_t, \qquad \epsilon_t \sim N(0, \sigma_\epsilon^2). \tag{12}$$

Given a previous approximate filter $\widehat{p}_{t-1}(x_{t-1})$, the one-step unnormalized density is

$$q_t(x_t, x_{t-1}) = \widehat{p}_{t-1}(x_{t-1}) \frac{1}{\sigma_\eta} \varphi\left(\frac{x_t - \rho x_{t-1}}{\sigma_\eta} \right) \frac{1}{\sigma_\epsilon} \varphi\left(\frac{y_t - x_t^2}{\sigma_\epsilon} \right), \tag{13}$$

where φ is the standard normal density.

If $y_t > 0$ and σ_ϵ is small, then

$$\frac{1}{\sigma_\epsilon} \varphi\left(\frac{y_t - x_t^2}{\sigma_\epsilon} \right) \tag{14}$$

has large values near both $x_t = \sqrt{y_t}$ and $x_t = -\sqrt{y_t}$. Thus a single observation can create two plausible state regions. The non-Gaussian shape is visible directly in the formula; it does not require any tensor argument.

Let ϕ_t approximate the square root:

$$\phi_t(x_t, x_{t-1}) \approx \sqrt{q_t(x_t, x_{t-1})}. \tag{15}$$

Define the nonnegative approximation

$$\widehat{q}_t(x_t, x_{t-1}) = \phi_t(x_t, x_{t-1})^2 + \tau_t \lambda_x(x_t) \lambda_{x^-}(x_{t-1}), \tag{16}$$

where

$$\tau_t \geq 0, \qquad \int \lambda_x(x_t)\,\mathrm{d}x_t = 1, \qquad \int \lambda_{x^-}(x_{t-1})\,\mathrm{d}x_{t-1} = 1. \tag{17}$$

The approximate normalizer is

$$\begin{aligned}
\widehat{Z}_t &= \iint \widehat{q}_t(x_t, x_{t-1})\,\mathrm{d}x_t\,\mathrm{d}x_{t-1} \\
&= \iint \phi_t(x_t, x_{t-1})^2\,\mathrm{d}x_t\,\mathrm{d}x_{t-1} + \tau_t \left[\int \lambda_x(x_t)\,\mathrm{d}x_t\right]\left[\int \lambda_{x^-}(x_{t-1})\,\mathrm{d}x_{t-1}\right] \\
&= \iint \phi_t(x_t, x_{t-1})^2\,\mathrm{d}x_t\,\mathrm{d}x_{t-1} + \tau_t.
\end{aligned} \tag{18}$$

The next approximate filter is

$$\begin{aligned}
\widehat{p}_t(x_t) &= \int \frac{\widehat{q}_t(x_t, x_{t-1})}{\widehat{Z}_t}\,\mathrm{d}x_{t-1} \\
&= \frac{1}{\widehat{Z}_t}\int \phi_t(x_t, x_{t-1})^2\,\mathrm{d}x_{t-1} + \frac{\tau_t}{\widehat{Z}_t}\lambda_x(x_t).
\end{aligned} \tag{19}$$

Why this matters. The defensive term is not mysterious. It contributes a controlled background density $(\tau_t/\widehat{Z}_t)\lambda_x(x_t)$ to the next filter. The main shape still comes from the square-root fit through $\int \phi_t^2\,\mathrm{d}x_{t-1}$.

Failure mode. If τ_t is too large, the defensive density can dominate the filter. If τ_t is too small and ϕ_t misses an important region, the filter can assign too little mass there. The equations show the tradeoff explicitly.

3 Tensor Trains As Low-Rank Coupling, Not Magic Compression

Start with two variables u_1, u_2 and basis functions $\{b_{1,a}\}_{a=1}^{n_1}$, $\{b_{2,b}\}_{b=1}^{n_2}$. A full coefficient expansion is

$$\phi(u_1, u_2) = \sum_{a=1}^{n_1}\sum_{b=1}^{n_2} F_{ab} b_{1,a}(u_1) b_{2,b}(u_2). \tag{20}$$

The coefficient matrix $F \in \mathbb{R}^{n_1 \times n_2}$ stores how coordinate u_1 couples to coordinate u_2. A rank-r factorization replaces

$$F_{ab} \approx \sum_{\ell=1}^{r} U_{a\ell} V_{\ell b}. \tag{21}$$

Substituting into (20) gives

$$\begin{aligned}
\phi(u_1, u_2) &\approx \sum_{\ell=1}^{r}\left[\sum_{a=1}^{n_1} U_{a\ell} b_{1,a}(u_1)\right]\left[\sum_{b=1}^{n_2} V_{\ell b} b_{2,b}(u_2)\right] \\
&= \sum_{\ell=1}^{r} U_\ell(u_1) V_\ell(u_2).
\end{aligned} \tag{22}$$

The storage changes from $n_1 n_2$ numbers to $r(n_1 + n_2)$ numbers.

For three variables, a full coefficient tensor

$$F_{abc}, \qquad 1 \le a \le n_1, \quad 1 \le b \le n_2, \quad 1 \le c \le n_3, \tag{23}$$

can be approximated by the chain

$$F_{abc} \approx \sum_{\ell_1=1}^{r_1} \sum_{\ell_2=1}^{r_2} G_1(a, \ell_1) G_2(\ell_1, b, \ell_2) G_3(\ell_2, c). \tag{24}$$

The corresponding function is

$$\phi(u_1, u_2, u_3) \approx G_1(u_1) G_2(u_2) G_3(u_3), \tag{25}$$

where $G_1(u_1)$ is a $1 \times r_1$ row, $G_2(u_2)$ is an $r_1 \times r_2$ matrix, and $G_3(u_3)$ is an $r_2 \times 1$ column. Multiplication produces a scalar.

The D-coordinate tensor train is

$$\phi(u_1, \ldots, u_D) = G_1(u_1) G_2(u_2) \cdots G_D(u_D), \tag{26}$$

$$G_k(u_k) = \sum_{a=1}^{n_k} C_{k,a} b_{k,a}(u_k), \qquad C_{k,a} \in \mathbb{R}^{r_{k-1} \times r_k}, \quad r_0 = r_D = 1. \tag{27}$$

The storage is roughly

$$\sum_{k=1}^{D} r_{k-1} n_k r_k, \tag{28}$$

instead of

$$\prod_{k=1}^{D} n_k. \tag{29}$$

Why this matters. For a chemistry or physics reader, the useful analogy is limited coupling. If a high-dimensional density can be described by local or moderately long-range interactions after a good coordinate ordering, the TT ranks may stay moderate. The TT is then not magic compression; it is a controlled low-rank description of how coordinate groups interact.

Failure mode. If every coordinate is strongly coupled to many distant coordinates, the ranks r_k can grow until (28) is no longer small. The TT representation remains mathematically valid, but it may stop being efficient.

4 From Square-Root Approximation To A Filter

The filtering density q_t is nonnegative. Therefore its square root is well-defined wherever q_t is finite:

$$s_t(u_t; \alpha) = \sqrt{q_t(u_t; \alpha)}. \tag{30}$$

Approximate s_t by a TT:

$$s_t(u_t; \alpha) \approx \phi_t(u_t; \alpha) = G_{t,1}(u_{t,1}; \alpha) \cdots G_{t,D}(u_{t,D}; \alpha). \tag{31}$$

Then define

$$\widehat{q}_t(u_t;\alpha) = \phi_t(u_t;\alpha)^2 + \tau_t(\alpha)\lambda_t(u_t), \qquad \lambda_t \geq 0, \quad \int \lambda_t = 1. \tag{32}$$

The normalizer and normalized joint density are

$$\widehat{z}_t(\alpha) = \int \widehat{q}_t(u_t;\alpha)\,\mathrm{d}u_t, \qquad \widehat{\pi}_t(u_t;\alpha) = \frac{\widehat{q}_t(u_t;\alpha)}{\widehat{z}_t(\alpha)}. \tag{33}$$

The approximate filter is the marginal

$$\widehat{p}_t(x_t,\vartheta;\alpha) = \int \widehat{\pi}_t(x_t,\vartheta,x_{t-1};\alpha)\,\mathrm{d}x_{t-1}. \tag{34}$$

To compute the square part of $\widehat{z}_t$, use the basis mass matrices

$$M_{t,k}[a,b] = \int b_{t,k,a}(u_k)b_{t,k,b}(u_k)\,\mathrm{d}\nu_{t,k}(u_k). \tag{35}$$

Starting from $R_{t,D} = 1$, define

$$R_{t,k-1} = \sum_{a,b=1}^{n_k} M_{t,k}[a,b]\,C_{t,k,a}R_{t,k}C_{t,k,b}^{\top}, \qquad k = D,\ldots,1. \tag{36}$$

Then

$$R_{t,0} = \int \phi_t(u_t;\alpha)^2\,\mathrm{d}u_t, \qquad \widehat{z}_t = R_{t,0} + \tau_t \tag{37}$$

for fixed normalized λ_t.

Why this matters. The square makes the approximation nonnegative. The mass contraction makes the integral computable without enumerating the full tensor grid. The normalization and marginalization then turn the approximation into a filter.

Failure mode. Signed or unconstrained low-rank density approximations can become negative. The squared-TT construction avoids that problem, but it can still be inaccurate if the square-root fit $\phi_t \approx \sqrt{q_t}$ is poor.

5 The Zhao–Cui Sequential Construction

The sequential construction can be written as the following recursion. Given $\widehat{p}_{t-1}$, form

$$q_t(x_t,\vartheta,x_{t-1}) = \widehat{p}_{t-1}(x_{t-1},\vartheta)f_\alpha(x_t \mid x_{t-1},\vartheta)g_\alpha(y_t \mid x_t,\vartheta). \tag{38}$$

Fit a square-root TT

$$\phi_t \approx \sqrt{q_t}. \tag{39}$$

Define

$$\widehat{q}_t = \phi_t^2 + \tau_t\lambda_t, \qquad \widehat{z}_t = \int \widehat{q}_t, \qquad \widehat{\pi}_t = \widehat{q}_t/\widehat{z}_t. \tag{40}$$

Propagate the filter by

$$\widehat{p}_t(x_t,\vartheta) = \int \widehat{\pi}_t(x_t,\vartheta,x_{t-1})\,\mathrm{d}x_{t-1}. \tag{41}$$

The same normalized joint density also defines conditional distributions. If the coordinates of u_t are ordered as $u_1, \dots, u_D$, then

$$\widehat{\pi}_t(u_1, \dots, u_D) = \widehat{\pi}_t(u_1)\widehat{\pi}_t(u_2 \mid u_1) \cdots \widehat{\pi}_t(u_D \mid u_1, \dots, u_{D-1}). \tag{42}$$

A conditional Knothe–Rosenblatt map samples these coordinates by inverting successive conditional cumulative distribution functions:

$$r_k = F_k(u_k \mid u_1, \dots, u_{k-1}), \qquad u_k = F_k^{-1}(r_k \mid u_1, \dots, u_{k-1}), \quad r_k \in (0, 1). \tag{43}$$

For the filtering and gradient results in this note, the map is secondary. What matters is that the same squared-TT object supplies marginals, conditionals, normalizers, and samples.

Zhao and Cui use this sequential squared-TT construction for state-space learning, including the nonseparable density over (x_t, θ, x_{t-1}), the squared density $\phi^2 + \tau\lambda$, and the marginal/conditional constructions [1]. Cui and Dolgov provide the squared inverse Rosenblatt transport machinery used to interpret the conditional maps [2].

Why this matters. The source papers do not merely provide a compression trick. They provide a way to use a nonnegative low-rank density representation inside the same normalization and marginalization operations that define Bayesian filtering.

Failure mode. The conditional maps inherit the quality of the squared-TT approximation. A bad density approximation yields bad marginal and conditional distributions even if the algebraic conditioning procedure is internally consistent.

6 Why Adaptive TT Filtering Is Not Automatically A Gradient Method

Let B denote all numerical branch choices:

$$B = \{\text{ranks, pivots, interpolation points, samples, truncations, factorization signs, shift rule}\}. \tag{44}$$

For a fixed branch B, the algorithm computes a scalar

$$\widehat{\ell}_T(\alpha; B) = \sum_{t=1}^{T} \{\log \widehat{z}_t(\alpha; B) - c_t(\alpha; B)\}. \tag{45}$$

An adaptive code chooses a branch from the parameter value:

$$B = B(\alpha). \tag{46}$$

Thus the adaptive value path is

$$L_{\text{adap}}(\alpha) = \widehat{\ell}_T(\alpha; B(\alpha)). \tag{47}$$

A fixed-branch derivative computes

$$\frac{\partial}{\partial \alpha_i} \widehat{\ell}_T(\alpha; B_0)\Big|_{\alpha=\alpha_0}, \qquad B_0 = B(\alpha_0), \tag{48}$$

not the derivative of (47) across branch changes.

Why this matters. Adaptation is useful for filtering because the representation can respond to the target density. A gradient, however, must differentiate one scalar formula. When the formula changes because $B(\alpha)$ changes, the derivative of the old branch is only a local branch derivative.

Failure mode. If finite-difference evaluations at $\alpha + he_i$ and $\alpha - he_i$ choose different ranks or interpolation points, then the finite difference is testing the adaptive value path, while the fixed-branch formula differentiates $\widehat{\ell}_T(\alpha; B_0)$. These are different mathematical objects.

7 The Fixed-Branch Filtering Algorithm

A fixed branch is specified before differentiation. For each time t, fix

$$B_t = (\text{ordering}, \text{domain maps}, \text{basis}, \text{ranks}, \text{fitting points}, \text{solver path}, \lambda_t, \tau_t, c_t, \text{factorization signs}). \tag{49}$$

The forward step is:

$$q_t = \widehat{p}_{t-1}^{\mathrm{TT}} f_\alpha g_\alpha, \qquad s_t = \exp(c_t/2)\sqrt{q_t}, \qquad \phi_t \approx s_t. \tag{50}$$

At fixed fitting points $u^{(j)}$, this means

$$s_j(\alpha) = \exp(c_t(\alpha)/2) \left[\widehat{p}_{t-1}^{\mathrm{TT}}(x_{t-1}^{(j)}, \vartheta^{(j)}; \alpha) f_\alpha(x_t^{(j)} \mid x_{t-1}^{(j)}, \vartheta^{(j)}) g_\alpha(y_t \mid x_t^{(j)}, \vartheta^{(j)})\right]^{1/2}. \tag{51}$$

One possible core update is interpolation:

$$A_{t,k}(\alpha) g_{t,k}(\alpha) = b_{t,k}(\alpha). \tag{52}$$

Another is fixed weighted least squares:

$$g_{t,k}(\alpha) = \arg\min_g \|W_{t,k}^{1/2}(A_{t,k}(\alpha) g - b_{t,k}(\alpha))\|_2^2. \tag{53}$$

The branch must choose the value path. The derivative later differentiates that path, not an unspecified mixture of paths.

After the cores are obtained, compute

$$R_{t,0} = \int \phi_t^2, \qquad \widehat{z}_t = R_{t,0} + \tau_t, \qquad \widehat{\ell}_t^{\mathrm{TT}} = \log \widehat{z}_t - c_t. \tag{54}$$

The normalized joint and filter are

$$\widehat{\pi}_t = \frac{\phi_t^2 + \tau_t \lambda_t}{\widehat{z}_t}, \qquad \widehat{p}_t^{\mathrm{TT}}(x_t, \vartheta) = \int \widehat{\pi}_t(x_t, \vartheta, x_{t-1})\, \mathrm{d}x_{t-1}. \tag{55}$$

Store the unnormalized marginal numerator

$$a_t(x_t, \vartheta) = \int \{\phi_t(x_t, \vartheta, x_{t-1})^2 + \tau_t \lambda_t(x_t, \vartheta, x_{t-1})\}\, \mathrm{d}x_{t-1}, \tag{56}$$

so that

$$\widehat{p}_t^{\mathrm{TT}}(x_t, \vartheta) = a_t(x_t, \vartheta)/\widehat{z}_t. \tag{57}$$

Why this matters. The saved numerator a_t is not bookkeeping clutter. It is needed because the next target q_{t+1} contains $\widehat{p}_t^{\mathrm{TT}}$, and therefore the gradient at $t+1$ needs the derivative of the previous approximate filter.

Failure mode. If the implementation saves only $\widehat{\ell}_t^{\mathrm{TT}}$ and discards the branch objects that produced it, then a later derivative implementation cannot prove that it differentiates the same scalar.

8 Proposition 1: Normalized Approximate Filtering

Assumption 1 (Fixed-branch filtering assumptions)**.** For every t, $\widehat{p}_{t-1}^{\mathrm{TT}}$ is nonnegative and integrates to one. The transition and likelihood densities are nonnegative and measurable. The fixed branch produces a measurable ϕ_t. The defensive density satisfies $\lambda_t \geq 0$ and $\int \lambda_t = 1$. The defensive mass satisfies $\tau_t \geq 0$. Finally,

$$0 < \widehat{z}_t = \int \{\phi_t(u)^2 + \tau_t \lambda_t(u)\}\,\mathrm{d}u < \infty. \tag{58}$$

Proposition 1 (Fixed-branch squared-TT filtering is normalized)**.** *Under the fixed-branch filtering assumptions, define*

$$\widehat{q}_t(u) = \phi_t(u)^2 + \tau_t \lambda_t(u), \qquad \widehat{\pi}_t(u) = \widehat{q}_t(u)/\widehat{z}_t, \tag{59}$$

and

$$\widehat{p}_t^{\mathrm{TT}}(x_t, \vartheta) = \int \widehat{\pi}_t(x_t, \vartheta, x_{t-1})\,\mathrm{d}x_{t-1}. \tag{60}$$

Then $\widehat{p}_t^{\mathrm{TT}}$ is a nonnegative normalized density. Starting from a normalized prior, the recursion defines normalized approximate filters for all times for which (58) *holds.*

Proof. Pointwise,

$$\widehat{q}_t(u) = \phi_t(u)^2 + \tau_t \lambda_t(u) \geq 0. \tag{61}$$

Because $0 < \widehat{z}_t < \infty$,

$$\widehat{\pi}_t(u) \geq 0, \qquad \int \widehat{\pi}_t(u)\,\mathrm{d}u = \frac{\int \widehat{q}_t(u)\,\mathrm{d}u}{\widehat{z}_t} = 1. \tag{62}$$

Using Tonelli's theorem for the nonnegative integrand,

$$\begin{aligned} \int \widehat{p}_t^{\mathrm{TT}}(x_t, \vartheta)\,\mathrm{d}x_t\,\mathrm{d}\vartheta &= \int\int \widehat{\pi}_t(x_t, \vartheta, x_{t-1})\,\mathrm{d}x_{t-1}\,\mathrm{d}x_t\,\mathrm{d}\vartheta \\ &= \int \widehat{\pi}_t(u)\,\mathrm{d}u = 1. \end{aligned} \tag{63}$$

Thus the marginal filter is nonnegative and normalized. The induction over time is immediate: a normalized prior gives the base case, and the displayed argument advances the property from $t-1$ to t. □

Why this matters. The proposition proves that the approximation is a proper probability density after every completed step. It does not prove that the density is accurate; that depends on the approximation quality of ϕ_t.

Failure mode. If $\widehat{z}_t = 0$, $\widehat{z}_t = \infty$, or the numerical branch creates a nonmeasurable or nonintegrable object, the normalization argument fails.

9 Proposition 2 In Layers: Same-Scalar Gradient

This section expands the gradient proof in layers. Each layer contributes one piece of the final derivative.

Layer A: derivative of the declared scalar

The fixed-branch scalar is

$$\widehat{\ell}_T^{\mathrm{TT}}(\alpha) = \sum_{t=1}^{T} \{\log \widehat{z}_t(\alpha) - c_t(\alpha)\}. \tag{64}$$

If $\widehat{z}_t(\alpha) > 0$, then

$$\partial_i \{\log \widehat{z}_t(\alpha) - c_t(\alpha)\} = \frac{\partial_i \widehat{z}_t(\alpha)}{\widehat{z}_t(\alpha)} - \partial_i c_t(\alpha). \tag{65}$$

Why this matters. The gradient problem reduces to computing $\partial_i \widehat{z}_t$ and $\partial_i c_t$ for the same branch that produced $\widehat{z}_t$ and c_t.

Layer B: derivative of the squared-TT normalizer

For fixed normalized λ_t,

$$\widehat{z}_t = \int \{\phi_t(u;\alpha)^2 + \tau_t(\alpha)\lambda_t(u)\}\,\mathrm{d}u. \tag{66}$$

Assuming differentiation may pass through the integral,

$$\begin{aligned} \partial_i \widehat{z}_t &= \int 2\phi_t(u;\alpha)\partial_i \phi_t(u;\alpha)\,\mathrm{d}u + \partial_i \tau_t(\alpha) \int \lambda_t(u)\,\mathrm{d}u \\ &= 2\int \phi_t(u;\alpha)\partial_i \phi_t(u;\alpha)\,\mathrm{d}u + \partial_i \tau_t(\alpha). \end{aligned} \tag{67}$$

If λ_t depends on α, the extra term

$$\tau_t \int \partial_i \lambda_t(u;\alpha)\,\mathrm{d}u \tag{68}$$

must be included.

Why this matters. The normalizer derivative depends on the square-root TT derivative, not on a separate derivative of a normalized filter. This is why the derivative path must track $\partial_i \phi_t$.

Layer C: derivative of the TT square root

The square-root TT is

$$\phi_t(u) = G_{t,1}(u_1)G_{t,2}(u_2)\cdots G_{t,D}(u_D). \tag{69}$$

The product rule gives

$$\partial_i \phi_t(u) = \sum_{k=1}^{D} G_{t,1}(u_1)\cdots \partial_i G_{t,k}(u_k)\cdots G_{t,D}(u_D). \tag{70}$$

With

$$G_{t,k}(u_k) = \sum_{a=1}^{n_k} C_{t,k,a} b_{t,k,a}(u_k), \tag{71}$$

one has, for fixed basis functions,

$$\partial_i G_{t,k}(u_k) = \sum_{a=1}^{n_k} (\partial_i C_{t,k,a}) b_{t,k,a}(u_k). \tag{72}$$

Why this matters. The derivative of the high-dimensional function is obtained by differentiating one core at a time. This is the same low-rank coupling idea as Section 3, now applied to sensitivities.

Layer D: derivative of the mass contraction

The forward mass contraction is

$$R_{t,D} = 1, \qquad R_{t,k-1} = \sum_{a,b} M_{t,k}[a,b] C_{t,k,a} R_{t,k} C_{t,k,b}^\top. \tag{73}$$

Assuming fixed mass matrices, the differentiated contraction is

$$\begin{aligned} \partial_i R_{t,k-1} = \sum_{a,b} M_{t,k}[a,b] \Big[& (\partial_i C_{t,k,a}) R_{t,k} C_{t,k,b}^\top + C_{t,k,a} (\partial_i R_{t,k}) C_{t,k,b}^\top \\ & + C_{t,k,a} R_{t,k} (\partial_i C_{t,k,b})^\top \Big], \end{aligned} \tag{74}$$

with $\partial_i R_{t,D} = 0$.

Why this matters. The scalar $R_{t,0}$ used in the forward likelihood is differentiated by the same backward contraction. This prevents the gradient from silently switching to another normalizer.

Layer E: derivative of the core solve

For an interpolation branch,

$$A_{t,k} g_{t,k} = b_{t,k}. \tag{75}$$

Differentiating gives

$$A_{t,k} \partial_i g_{t,k} = \partial_i b_{t,k} - (\partial_i A_{t,k}) g_{t,k}. \tag{76}$$

For a weighted least-squares branch,

$$g_{t,k} = \arg\min_g \| W_{t,k}^{1/2} (A_{t,k} g - b_{t,k}) \|_2^2, \tag{77}$$

the normal equations are

$$N_{t,k} g_{t,k} = A_{t,k}^\top W_{t,k} b_{t,k}, \qquad N_{t,k} = A_{t,k}^\top W_{t,k} A_{t,k}. \tag{78}$$

With fixed $W_{t,k}$,

$$\begin{aligned} N_{t,k} \partial_i g_{t,k} = & (\partial_i A_{t,k})^\top W_{t,k} (b_{t,k} - A_{t,k} g_{t,k}) \\ & + A_{t,k}^\top W_{t,k} (\partial_i b_{t,k} - (\partial_i A_{t,k}) g_{t,k}). \end{aligned} \tag{79}$$

Why this matters. The derivative of a core is not guessed. It is obtained by differentiating the same linear algebra problem that produced the core.

Layer F: derivative of the previous filter

The target at time t contains the previous filter:

$$q_t = \widehat{p}_{t-1}^{\mathrm{TT}} f_\alpha g_\alpha. \tag{80}$$

Where the factors are positive,

$$\partial_i \log q_t = \partial_i \log \widehat{p}_{t-1}^{\mathrm{TT}} + \partial_i \log f_\alpha + \partial_i \log g_\alpha. \tag{81}$$

Using the stored numerator from (56),

$$\widehat{p}_{t-1}^{\mathrm{TT}}(v;\alpha) = \frac{a_{t-1}(v;\alpha)}{\widehat{z}_{t-1}(\alpha)}, \tag{82}$$

so

$$\partial_i \log \widehat{p}_{t-1}^{\mathrm{TT}}(v;\alpha) = \frac{\partial_i a_{t-1}(v;\alpha)}{a_{t-1}(v;\alpha)} - \frac{\partial_i \widehat{z}_{t-1}(\alpha)}{\widehat{z}_{t-1}(\alpha)}. \tag{83}$$

The derivative $\partial_i a_{t-1}$ is another marginal contraction, with one previous TT core differentiated at a time.

Why this matters. The gradient is recursive because the filter is recursive. Time t cannot be differentiated correctly unless the derivative of the time $t-1$ filter is carried forward.

Assumption 2 (Fixed-branch differentiability)**.** The fixed-branch filtering assumptions hold. The branch choices are fixed. The model densities, previous-filter numerator, coordinate maps, basis maps, core coefficients, defensive mass, and stabilizing constant are differentiable in α_i. The displayed differentiations may pass through the finite contractions and integrals. The fixed interpolation or least-squares systems are nonsingular, or a differentiable pseudoinverse branch is declared.

Proposition 2 (Same-scalar fixed-branch gradient)**.** *Under the fixed-branch differentiability assumptions, the derivative of*

$$\widehat{\ell}_T^{\mathrm{TT}}(\alpha) = \sum_{t=1}^{T} \{\log \widehat{z}_t(\alpha) - c_t(\alpha)\} \tag{84}$$

is

$$\partial_i \widehat{\ell}_T^{\mathrm{TT}} = \sum_{t=1}^{T} \left[\frac{\partial_i R_{t,0} + \partial_i \tau_t}{R_{t,0} + \tau_t} - \partial_i c_t \right], \tag{85}$$

for the direct mass-contraction branch with fixed normalized λ_t. If λ_t is parameter-dependent, the additional term in (68) *must be included.*

Proof. Layer A proves the derivative of each log-normalizer contribution. Layer B expresses $\partial_i \widehat{z}_t$ in terms of ϕ_t, $\partial_i \phi_t$, and $\partial_i \tau_t$. Layer C gives $\partial_i \phi_t$ by differentiating one TT core at a time. Layer D shows that the derivative of the square mass $R_{t,0}$ is computed by the same mass-contraction recursion used in the forward pass, with product-rule terms. Layer E gives the core sensitivities for the fixed interpolation or fixed least-squares branch. Layer F supplies the recursive derivative of the previous approximate filter inside q_t. Substituting $\widehat{z}_t = R_{t,0} + \tau_t$ into Layer A gives (85). □

Failure mode. If the rank, pivot set, fitting points, branch signs, or least-squares system change with α, the derivative above remains the derivative of the frozen local branch, not of the adaptive code path.

10 Algorithmic Gradient Recipe

Inputs. For each t, provide

$$B_t, \quad \widehat{p}_{t-1}^{\mathrm{TT}}, \quad f_\alpha, \quad g_\alpha, \quad \partial_i \log f_\alpha, \quad \partial_i \log g_\alpha, \quad y_t. \tag{86}$$

The branch data B_t includes bases, ranks, fitting points, mass matrices, linear-system type, defensive density, defensive mass, shift rule, and factorization signs.

Forward pass. For $t = 1, \ldots, T$:

$$q_t = \widehat{p}_{t-1}^{\mathrm{TT}} f_\alpha g_\alpha, \qquad s_t = \exp(c_t/2)\sqrt{q_t}. \tag{87}$$

Solve either (52) or (53) for the TT cores. Compute

$$R_{t,0}, \quad \widehat{z}_t = R_{t,0} + \tau_t, \quad \widehat{\ell}_t^{\mathrm{TT}} = \log \widehat{z}_t - c_t, \tag{88}$$

and store a_t from (56).

Sensitivity pass. For each parameter coordinate α_i and time t:

$$\partial_i \log q_t = \partial_i \log \widehat{p}_{t-1}^{\mathrm{TT}} + \partial_i \log f_\alpha + \partial_i \log g_\alpha. \tag{89}$$

Differentiate the chosen core solve, producing $\partial_i C_{t,k,a}$. Propagate $\partial_i R_{t,k}$ backward with (74). Form

$$\partial_i \widehat{z}_t = \partial_i R_{t,0} + \partial_i \tau_t \tag{90}$$

and

$$\partial_i \widehat{\ell}_t^{\mathrm{TT}} = \frac{\partial_i \widehat{z}_t}{\widehat{z}_t} - \partial_i c_t. \tag{91}$$

Carry the filter sensitivity forward through

$$\partial_i(\widehat{q}_t/\widehat{z}_t) = \frac{\partial_i \widehat{q}_t}{\widehat{z}_t} - \frac{\widehat{q}_t\, \partial_i \widehat{z}_t}{\widehat{z}_t^2}, \tag{92}$$

then marginalize over x_{t-1}.

Finite-difference parity. For a frozen branch B_0, check

$$\frac{\widehat{\ell}_T^{\mathrm{TT}}(\alpha + he_i; B_0) - \widehat{\ell}_T^{\mathrm{TT}}(\alpha - he_i; B_0)}{2h} \approx \partial_i \widehat{\ell}_T^{\mathrm{TT}}(\alpha; B_0). \tag{93}$$

If the branch is not frozen, this test no longer checks the same derivative.

Failure mode. The most dangerous implementation error is a value-gradient mismatch: the forward pass computes one scalar, while the derivative pass differentiates a slightly different scalar. The parity test is designed to catch that error.

11 What The Construction Gives And What Must Be Tested

Mathematically, the construction gives

$$\widehat{q}_t = \phi_t^2 + \tau_t \lambda_t, \qquad \widehat{z}_t = \int \widehat{q}_t, \qquad \widehat{p}_t^{\mathrm{TT}} = \int \widehat{q}_t / \widehat{z}_t \, \mathrm{d}x_{t-1}. \tag{94}$$

Proposition 1 proves that this is a normalized approximate filter. Proposition 2 proves that, on a fixed branch,

$$\nabla_\alpha \sum_t \{\log \widehat{z}_t - c_t\} \tag{95}$$

is computed by differentiating the same branch.

What must still be tested is numerical:

$$\|\widehat{q}_t - q_t\|, \qquad \max_k r_{t,k}, \qquad \kappa(A_{t,k}) \text{ or } \kappa(N_{t,k}), \qquad \text{finite-difference parity error.} \tag{96}$$

These quantities answer different questions. Approximation error tests whether the filter is accurate. Rank growth tests whether the TT idea is efficient. Conditioning tests whether the linear algebra is stable. Parity tests whether the implemented gradient differentiates the same scalar.

Why this matters. The derivation makes Zhao–Cui-style TT filtering a credible candidate because it defines a normalized approximate filter and a fixed-branch gradient. It does not by itself show that the candidate wins on the target model. That requires numerical experiments and implementation tests.

A Source And Code Anchors

The derivations above are written in BayesFilter notation. The source anchors below record where the corresponding objects appear in the inspected sources.

- Zhao–Cui equations (1)–(3): state-space model notation.
- Zhao–Cui equations (9)–(12) and Algorithm 1: sequential density over (x_t, θ, x_{t-1}), TT approximation, and integration architecture.
- Zhao–Cui equation (13) and nearby Lemma 1: squared-TT density $\phi^2 + \tau\lambda$.
- Zhao–Cui Algorithm 2: sequential use of squared-TT approximations.
- Zhao–Cui Propositions 2 and 4: marginal and conditional constructions.
- Zhao–Cui Section 4.1: normalizer/evidence discussion.
- Cui–Dolgov Sections 2–3: squared inverse Rosenblatt and functional TT transport substrate.
- Inspected MATLAB snapshot: the scalar path adds $\log(\texttt{sirt.z}) - \texttt{const}$, and the TT marginalization stores a normalizer of the form square-mass plus defensive mass. These code references show consistency of scalar structure; the mathematical justification is the derivation in the main text.

References

[1] Zhao, Y. and Cui, T. (2024). Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25(244):1–51.

[2] Cui, T. and Dolgov, S. (2022). Deep composition of tensor-trains using squared inverse Rosenblatt transports. *Foundations of Computational Mathematics*, 22:1863–1922. DOI: 10.1007/s10208-021-09537-5.